



Looping Adventures in Python

Section 1: Learn

What is Looping?

Looping allows a program to **repeat a block of code multiple times** until a specific condition is met.

Why Use Loops?

- **Avoids Repetitive Code:** Reduces duplication and makes code cleaner.
- **Automates Repetitive Tasks:** Useful in **data processing, automation, and AI**.
- **Enhances Efficiency:** Saves time and makes programs faster.

Types of Loops in Python:

1. **for Loop:** Iterates over a sequence (list, tuple, string, range, etc.).
2. **while Loop:** Repeats execution as long as a condition is **True**.
3. **Nested Loops:** Loops inside loops for multi-level iteration.
4. **Loop Control Statements** (**break, continue, pass**): Modify loop execution.

Fun Fact: How Were Loops Used Before High-Level Languages?

Early programming required **manual jump instructions** (**GOTO** statements) to repeat tasks. Loops **automated this**, making coding **simpler and more readable!**



Section 2: Practice

1. The **for** Loop

Example: Print Numbers from 1 to 5

```
for i in range(1, 6):  
    print(i)
```

Output:

```
1  
2  
3  
4  
5
```

Iterating Over a List

```
fruits = ["Apple", "Banana", "Cherry"]  
for fruit in fruits:  
    print(fruit)
```

Output:

```
Apple  
Banana  
Cherry
```



Problem: Print Even Numbers from 1 to 10

```
for num in range(1, 11):  
    if num % 2 == 0:  
        print(num)
```

Output:

```
2  
4  
6  
8  
10
```

2. The **while** Loop

Example: Countdown from 5 to 1

```
count = 5  
while count > 0:  
    print("Counting:", count)  
    count -= 1
```

Output:

```
Counting: 5  
Counting: 4  
Counting: 3  
Counting: 2  
Counting: 1
```



Problem: Keep Asking for Input Until a User Enters "exit"

```
user_input = ""  
while user_input != "exit":  
    user_input = input("Enter something (type 'exit' to stop): ")  
    print(f"You entered: {user_input}")
```

Output (Example User Input):

```
Enter something (type 'exit' to stop): hello  
You entered: hello  
Enter something (type 'exit' to stop): Python  
You entered: Python  
Enter something (type 'exit' to stop): exit  
You entered: exit
```

3. Nested Loops

Example: Multiplication Table (2 x 2 Grid)

```
for i in range(1, 3):  
    for j in range(1, 3):  
        print(f"{i} x {j} = {i * j}")
```

Output:

```
1 x 1 = 1  
1 x 2 = 2  
2 x 1 = 2  
2 x 2 = 4
```



4. Loop Control Statements

break: Exit Loop Immediately

```
for num in range(1, 6):  
    if num == 3:  
        break  
    print(num)
```

Output:

```
1  
2
```

continue: Skip Current Iteration

```
for num in range(1, 6):  
    if num == 3:  
        continue  
    print(num)
```

Output:

```
1  
2  
4  
5
```



pass: Do Nothing (Placeholder for Future Code)

```
for i in range(5):  
    pass # No action performed, avoids syntax errors
```

Try This: FizzBuzz Challenge

Print numbers from 1 to 15:

- If a number is divisible by 3, print "Fizz".
- If a number is divisible by 5, print "Buzz".
- If divisible by both 3 and 5, print "FizzBuzz".

```
for num in range(1, 16):  
    if num % 3 == 0 and num % 5 == 0:  
        print("FizzBuzz")  
    elif num % 3 == 0:  
        print("Fizz")  
    elif num % 5 == 0:  
        print("Buzz")  
    else:  
        print(num)
```

Output:

```
1  
2  
Fizz  
4  
Buzz
```



```
Fizz
7
8
Fizz
Buzz
11
Fizz
13
14
FizzBuzz
```

Section 3: Know More (FAQs)

Q1: What is the difference between **for and **while** loops?**

Loop Type	Use Case
for loop	Used when the number of iterations is known
while loop	Used when the loop runs until a condition is met

Example:

```
# for loop (Fixed repetitions)
```

```
for i in range(5):
```

```
    print(i)
```

```
# while loop (Unknown repetitions)
```



```
x = 5
while x > 0:
    print(x)
    x -= 1
```

Q2: What is an infinite loop?

Answer: A loop that **never stops running** due to an incorrect condition.

Example:

```
while True:
    print("This will run forever!")
```

Q3: How do you exit a loop manually?

Use **break**:

```
for i in range(10):
    if i == 5:
        break
    print(i)
```

Output:

```
0
1
2
3
4
```



Q4: How to iterate over a dictionary using loops?

```
student = {"name": "Amit", "age": 21, "course": "Python"}

for key, value in student.items():
    print(f"{key}: {value}")
```

Output:

```
name: Amit
age: 21
course: Python
```

Q5: What is the difference between **break** and **continue**?

Statement	Purpose
break	Exits the loop immediately
continue	Skips current iteration and continues to next

Example:

```
for num in range(1, 6):
    if num == 3:
        break # Stops loop at 3
    print(num)

for num in range(1, 6):
    if num == 3:
        continue # Skips printing 3
```



```
print(num)
```

Loops **simplify repetitive tasks**, making Python **efficient and powerful**. 🚀

Keep practicing to master **real-world automation and problem-solving**!